

Tendências de Motores de Jogos

Semana 7

SaveGame, Integração e Encerramento do Módulo 2

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 2 — Construir Sistemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender a serialização e recuperação de estado como problema universal de persistência entre sessões
- Implementar `BP_SaveGame` e `SaveComponent` , centralizando gravação e leitura do progresso de coleta de itens
- Integrar, em um único fluxo jogável, todos os sistemas do Módulo 2 (GameMode a SaveGame)
- Justificar tecnicamente, em Code Review, as decisões de arquitetura adotadas

Como a semana está organizada



Encontro 1

SaveGame Object — BP_SaveGame e SaveComponent



Encontro 2

Integração final do Módulo 2 — Code Review e Playtest coletivo

Tendências de Motores de Jogos

ENCONTRO 1

SaveGame Object

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



**Se você fechar o jogo agora e reabrir, o que
sobra do progresso do jogador?**

Pense no que existe apenas enquanto o jogo está rodando.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Persistência de estado: o problema universal

- Todo progresso do jogador existe apenas em memória enquanto o jogo roda
- Ao fechar o editor ou o build, esse estado desaparece
- Toda engine madura precisa capturar uma fração do estado e gravá-la em formato recuperável
- A Unreal resolve isso com uma classe dedicada, fora do fluxo normal de Actors

DICA

Um SaveGame Object não existe no nível, não é renderizado e não recebe Tick — sua única responsabilidade é ser serializado.

SaveComponent: ponto único de acesso

- Centraliza toda leitura/escrita de `BP_SaveGame` em um único Actor Component
- Evita que cada Actor de coleta implemente sua própria lógica de Save/Load
- Qualquer sistema futuro (como o Inventário, na Semana 10) depende de uma única função exposta
- Mesmo princípio de responsabilidade única já aplicado ao `InteractionComponent` da Semana 5

NA INDÚSTRIA

Nenhum Actor deve implementar sua própria lógica de leitura/escrita de arquivo — essa responsabilidade é sempre centralizada.

Estado em memória × Estado serializado



Persistência de estado: Unreal × Unity

Unreal

Classe dedicada `USaveGame`, com suporte nativo a serialização binária de propriedades marcadas

Unity

`PlayerPrefs` para dados simples, ou serialização própria em JSON (`JsonUtility`) para estados mais complexos — o desenvolvedor define o formato

Demonstração: BP_SaveGame

O professor cria `BP_SaveGame` a partir da classe `SaveGame`, com uma propriedade `ItensColetados` (Array de Name), guardando os identificadores dos itens já coletados.

Resultado esperado: classe simples, apenas dados, sem lógica de gameplay.

Demonstração: SaveComponent

O professor cria `SaveComponent` (Actor Component) com as funções `SalvarProgresso` e `CarregarProgresso`, adiciona o componente ao `BP_Player` e conecta a gravação ao Event Dispatcher de coleta já existente.

Por que: provar que a gravação ocorre no momento certo — ao coletar — sem duplicar lógica em cada Actor.

Imagem sugerida

Objetivo: mostrar a diferença entre estado em memória (runtime) e estado serializado (SaveGame Object), reforçando o papel do SaveComponent como ponto único de acesso.

Enquadramento: diagrama de duas colunas lado a lado, conectadas por um único ponto central.

Elementos presentes: coluna esquerda — "Estado em memória (Actors, variáveis em runtime)"; coluna direita — "Estado serializado (BP_SaveGame em disco)"; centro — ícone de engrenagem rotulado "SaveComponent".

Destaque visual: todas as setas de gravação e leitura passam pelo bloco central do SaveComponent, nunca diretamente entre Actors e o arquivo salvo.

Legenda sugerida: "Um único ponto de passagem entre o jogo em execução e o progresso salvo em disco."

Arquitetura: roadmap atualizado

- `BP_SaveGame` e `SaveComponent` — novas classes na subpasta `Blueprints/Components/`
- `SaveComponent` adicionado ao `BP_Player`, conectado ao Event Dispatcher da Semana 5
- Linha "SaveComponent / BP_SaveGame" do roadmap concluída (PROJECT_ARCHITECTURE.md, seção 6)

NA INDÚSTRIA

Nenhum sistema futuro precisará reimplementar Save/Load — todos passarão pelo `SaveComponent` já existente.

Boas práticas

✓ BOAS PRÁTICAS

- Manter `BP_SaveGame` como estrutura de dados simples — sem lógica de gameplay
- Verificar duplicatas antes de adicionar um item ao array `ItensColetados`
- Nomear o slot de save de forma explícita e centralizada em um único lugar



Laboratório do Encontro 1

Implementar `BP_SaveGame` e `SaveComponent`, aplicados ao próprio conjunto de itens de `DT_Items` de cada grupo, validando com um teste de fechar e reabrir o nível.

OBJETIVOS

Critério de sucesso: progresso de coleta gravado e recuperado corretamente, com nomenclatura conforme `PROJECT_ARCHITECTURE.md`.

Fechando o Encontro 1

- SaveGame Object resolve persistência de estado entre sessões; SaveComponent centraliza o acesso
- BP_SaveGame e SaveComponent funcionais, integrados ao Event Dispatcher da Semana 5
- Próximo encontro: integrar tudo o que foi construído desde a Semana 4 em um único fluxo jogável

Tendências de Motores de Jogos

ENCONTRO 2

Integração, Code Review e Playtest

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro

- Revisão integrada de todos os sistemas do Módulo 2
- Laboratório de integração em um único fluxo jogável
- Code Review em cada grupo
- Playtest coletivo com rodízio entre grupos

Um Vertical Slice não é a soma de sistemas isolados

- GameMode e GameState fornecem o contexto de partida
- PlayerController e GameInstance fazem a ponte entre jogador e persistência
- `BPI_Interactable` e Event Dispatchers desacoplam a comunicação entre Player e mundo
- `DT_Items` separa dado de lógica; SaveGame garante que tudo sobreviva entre sessões



DICA

Um Vertical Slice funcional é a integração coerente entre sistemas — nunca a soma de peças isoladas.

Sistemas do Módulo 2 integrados



Revisão integrada: antes de integrar

- Confirmar que cada Blueprint da lista compila sem erros
- Testar isoladamente, em modo Play, cada objeto interativo já implementado
- Anotar qualquer sistema que não esteja mais funcionando desde o último teste (regressão)

NA INDÚSTRIA

Tratar esta revisão como um smoke test antes de uma entrega em um estúdio profissional: rápido, mas cobrindo todos os sistemas críticos.

Laboratório de integração

- Organizar em um único percurso navegável os objetos interativos existentes (porta/alavanca, baús/itens, checkpoints)
- Confirmar que cada coleta aciona `SalvarProgresso`, e que o carregamento do nível aciona `CarregarProgresso`
- Testar o percurso completo, do início ao fim, em uma única sessão de Play
- Fechar e reabrir o nível, confirmando que o percurso completo é recuperado

O que fazer quando um conflito aparecer

- Isolar qual sistema não está se comunicando corretamente
- Revisar se a comunicação passa pelos padrões corretos: Interface, Event Dispatcher ou SaveComponent
- Corrigir apenas o ponto de falha — nunca recriar o sistema do zero
- Documentar pendências não resolvidas a tempo, para apresentação transparente no Code Review

ATENÇÃO

Sistemas funcionais isoladamente, mas nunca testados em conjunto, é o problema mais esperado deste encontro.

O que muda entre engines nesta integração

Unreal

GameMode/GameState, Interfaces, Event Dispatchers, Data Table e SaveGame como classes nativas dedicadas

Unity

Managers/Singletons por convenção, Interfaces em C#, UnityEvent/Actions, ScriptableObject e PlayerPrefs/JSON

Code Review: avaliação por dentro

- Verifica organização, nomenclatura, modularidade, reutilização e comunicação desacoplada
- Cada grupo explica por que cada decisão de arquitetura foi tomada
- Exemplo de pergunta: por que um Actor usa Interface em vez de referência direta?
- Conforme Rubrica 4 do Sistema de Avaliação

DICA

O Code Review espera que o grupo relacione a decisão ao conceito universal por trás dela, não apenas ao resultado funcional.

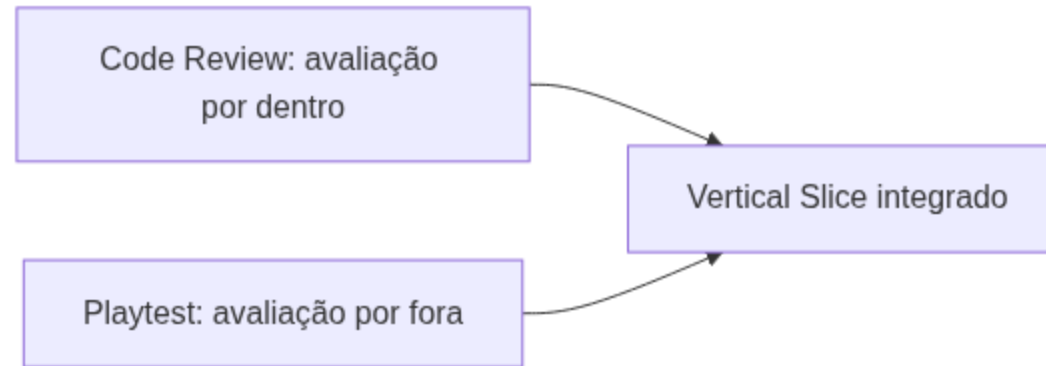
Playtest coletivo: avaliação por fora

- Um jogador externo ao grupo testa o percurso completo, sem explicação prévia além do mínimo necessário
- Registra funcionamento, usabilidade, bugs, feedback visual e clareza de interface
- O grupo observa e evita intervir, exceto em caso de bloqueio real
- Conforme Rubrica 5 do Sistema de Avaliação

⚠ ATENÇÃO

A própria equipe jogar o Playtest invalida o instrumento — confunde "funciona para quem desenvolveu" com "funciona para um jogador novo".

Duas lentes sobre o mesmo Vertical Slice



Boas práticas

✓ BOAS PRÁTICAS

- Usar Comment Boxes nos pontos de integração entre sistemas de módulos diferentes
- Preparar o projeto para o Code Review: Blueprints organizados, comentados, sem lógica duplicada visível
- Identificar o jogador externo do Playtest com antecedência, antes do rodízio começar

Laboratório do Encontro 2

Integrar portas/alavancas, baús/itens e SaveGame em um único percurso jogável, testado do início ao fim, incluindo fechar e reabrir o nível.

OBJETIVOS

Critério de sucesso: progresso completo persistindo corretamente entre sessões, para todos os objetos do percurso — não apenas um item isolado.

Desafio: justificar a arquitetura

Cada grupo apresenta sua integração completa do Módulo 2, justificando tecnicamente as escolhas de arquitetura adotadas — uso de Interfaces, Event Dispatchers, Data Table e centralização da persistência no `SaveComponent`.

ATENÇÃO

Sem desafio de liberdade de solução — o objetivo é justificar decisões já tomadas ao longo do módulo, não propor uma nova.

Resumo da Semana 7

- SaveGame Object e SaveComponent resolvem persistência de estado entre sessões
- Todos os sistemas do Módulo 2 (GameMode a SaveGame) integrados em um único fluxo jogável
- Code Review e Playtest coletivo aplicados pela primeira vez na disciplina
- Unidade II — Construir Sistemas — encerrada

Checklist Final do Módulo 2

OBJETIVOS

- GameMode, GameState, PlayerController e GamelInstance funcionais
- BPI_Interactable e Event Dispatchers em pelo menos um objeto interativo
- DT_Items tipada por S_ItemData / E_ItemType , aplicada a um Actor de coleta
- BP_SaveGame / SaveComponent persistindo o progresso entre sessões
- Code Review e Playtest coletivo concluídos, com pendências registradas quando houver

Próximos passos

DICA

A Semana 8 abre a Unidade III — Resolver Problemas, com Animation Blueprint, Blend Spaces e Montages atuando sobre o `BP_Player` já consolidado. A metodologia muda para Challenge Based Learning: a partir de agora, o professor apresenta problemas e os grupos propõem soluções com autonomia crescente, sem tutoriais passo a passo.

Leitura recomendada: Saving and Loading Your Game (Epic Games Documentation).